

ShepherdSoft → Vue 3 + NestJS Migration Plan

Context

ShepherdSoft is a church management system (Branches, Members, Guests, Follow-ups, Families) currently built on **.NET 10 + Aspire** with six projects (Web MVC, Api, DBL/Dapper, Gateway, AppHost, ServiceDefaults) talking to **MySQL** via Dapper + stored procedures. The team has already begun a Vue migration — CORS is wired for `http://localhost:5173` and a Tailwind v4 + KTUI pipeline exists inside the MVC project.

This plan covers a **branch-user-side cutover** to a new Vue 3 + TypeScript + NestJS stack rooted at `D:\Projects\Vue\ShepherdSoft-TS`. The .NET Admin Area (and the `ShepherdSoft.Api` admin controllers) stay in place for now; the new stack replaces the regular-user MVC views (`ShepherdSoft/Views/Auth|Home|Guest|Membership|Messaging`) and the four branch-user API controllers.

Locked Decisions

Topic	Choice	Why
Backend framework	NestJS	Mirrors existing layered .NET (Controllers/Services/Repositories) with DI + decorators
ORM	Prisma (introspected from existing MySQL)	Type-safe queries; introspect avoids schema drift
Stored procedures	Keep all MySQL SPs , call via <code>prisma.\$queryRawUnsafe / \$executeRawUnsafe</code>	Lowest behavioral risk; SPs hold non-trivial workflows
API contract	Preserve the action-code envelope (<code>POST /api/v1/<area>/service</code> with <code>{tsp, ver, act, content, caller}</code> → <code>{stat, msg, data}</code>)	Drop-in for any in-flight Vue work; matches <code>HTTP_API_ACTION</code> codes verbatim
Rollout	Big-bang for the branch-user surface	Build complete, verify parity, then switch DNS/proxy. .NET admin stays untouched.
SPA scope	Branch-user pages only (<code>/auth, /, /guest/*, /membership/*, /messaging/*</code>)	Admin Area stays .NET MVC
Auth storage	JWT in HttpOnly Secure cookie (<code>sameSite=lax</code>) set by NestJS	XSS-resistant; matches the existing Cookie auth feel for users
DB connection	DATABASE_URL in backend/.env	Don't reimplement the Bmat.Tools <code>DbConnData</code> decryption. <code>.env.example</code> committed, <code>.env</code> git-ignored

Scope

In scope (must be re-implemented):

- API controllers: `ShepherdSoft.Api/Controllers/AuthController.cs`, `DataController.cs`, `GuestController.cs`, `MemberController.cs`, `BaseController.cs`
- Action codes: `DATA_GET_LIST` (1), `DATA_GET_LIST_GROUP` (2), `MEMBER_FIND/GET/ADD/FAM_*` (100–105), `GUEST_*` (200–209), `AUTH_CHANGE_PASS` (101)
- Branch-user views: `Auth/Login`, `Auth/ChangePass`, `Home/Index`, `Home/Privacy`, `Guest/Index|Register|Guest|FollowUps`, `Membership/Index|Member|Depts|Flsps|Fams`, `Messaging/Index`
- Partial → Vue modal components: `_AddGuest`, `_AddFollowUp`, `_CompleteFollowUp`, `_PromoteGuest`, `_AddMember`, `_AddFam`
- `WebApiDataHandler` middleware behavior (injects caller context from JWT into `ApiRequestModel.CallerData`)
- `EmailService` (MailKit → Nodemailer) — only the welcome-user path actually fires from branch-user flows; admin still uses .NET. Likely **not needed in scope** but the helper should exist.

Out of scope (left untouched on .NET):

- Everything in `ShepherdSoft.Api/Areas/Admin/Controllers/*` (`AuthController`, `BranchController`, `UserController`) and corresponding `ADMIN_API_ACTIONS`
- `ShepherdSoft/Areas/Admin/*` MVC views (`Branch/Index`, `Branch/Users`, `Branch/ManageMember`, `User/Index`, `Ref/*`, `Me/Index`)
- `ShepherdSoft.Gateway`, `ShepherdSoft.AppHost`, `ShepherdSoft.ServiceDefaults`
- The MySQL schema, views (`vw_Guests`, `vw_Members`, `vw_Branches`, `vw_BranchUsers`, `vw_Fams`, `vw_GuestFollowUps`), stored procedures, sequences, seed data — **all reused as-is**

Target Architecture

```

D:\Projects\Vue\ShepherdSoft-TS\
├─ package.json
├─ ["frontend","backend","packages/*"]
├─ pnpm-workspace.yaml
├─ tsconfig.base.json
├─ .editorconfig .gitignore .nvmrc
├─ README.md
├─ docker-compose.yml
└─ at existing schema)

├─ packages/
│   └─ shared/
│       ├── package.json
│       └─ src/
│           └─ api/
│               ├── envelope.ts
│               ├── actions.ts
│               └─ status.ts
└─ (mirrors ApiModels.cs)

(1,2,100..105,200..209)

```

← pnpm workspaces monorepo
 ← workspaces:
 ← run instructions
 ← optional: mysql for local dev (points
 ← shared TS types & action-code constants
 ← name: @shepherd/shared
 ← ApiRequest, ApiResponse, RequestHeader
 ← HTTP_API_ACTION enum
 ← Status: Ok=0, Error=1

3 / 12

```

| | | | | security.sp.ts      ← sp_VerifyUser, sp_UserLogin,
sp_ChangeUserPwd
| | | | | member.sp.ts       ← sp_CreateMember, sp_CreateFamily
| | | | | guest.sp.ts        ← sp_CreateGuest,
sp_PromoteGuestToMember, sp_DeleteGuest
| | | | | followup.sp.ts     ← sp_AddGuestFollowUp,
sp_CompleteGuestFollowUp, sp_CancelGuestFollowUp
| | | | | data.sp.ts         ← bfp_GetListData, bfp_GetListGroup,
fn_GenMemberNo
| | | | | auth/              ← module
| | | | | auth.module.ts
| | | | | auth.controller.ts  ← POST /api/v1/auth/login,
/api/v1/auth/service (logout, change-pass)
| | | | | auth.service.ts     ← password verify (PBKDF2 mirroring
SecurityManager), JWT issue, session row
| | | | | token.service.ts    ← mirrors
ShepherdSoft.Api/Services/TokenService.cs (claims: sub, jti, iat, userData)
| | | | | crypto.ts          ← password hash/verify (must match
existing salt+hash format)
| | | | | strategies/
| | | | |   jwt.strategy.ts
| | | | | data/              ← module
| | | | |   data.module.ts
| | | | |   data.controller.ts ← POST /api/v1/data/service
| | | | |   data.service.ts   ← act=1 list, act=2 list-group
| | | | | guest/            ← module
| | | | |   guest.module.ts
| | | | |   guest.controller.ts ← POST /api/v1/guest/service
| | | | |   guest.service.ts  ← dispatches 200..209
| | | | |   guest.repository.ts ← Prisma queries against guests +
vw_Guests
| | | | | followup.repository.ts ← vw_GuestFollowUps + SPs
| | | | | member/           ← module
| | | | |   member.module.ts
| | | | |   member.controller.ts ← POST /api/v1/member/service
| | | | |   member.service.ts  ← dispatches 100..105
| | | | |   member.repository.ts ← vw_Members
| | | | |   family.repository.ts ← vw_Fams
| | | | | mail/
| | | | |   mail.module.ts
| | | | |   mail.service.ts   ← nodemailer (smtp.gmail.com, app
password from env)
| | | | | logging/
| | | | |   logger.service.ts  ← pino, json structured (replaces LogWiz;
LogWiz log files still produced by .NET)
| | | | | test/
| | | | |   auth.e2e-spec.ts
| | | | |   guest.e2e-spec.ts
| | | | |   member.e2e-spec.ts
| | | | | frontend/        ← Vite + Vue 3
| | | | |   package.json     ← vue, vue-router, pinia, axios,
@vueuse/core, vee-validate + zod, dayjs
| | | | |   vite.config.ts    ← server.port=5173, proxy /api →

```

```

http://localhost:3000
├─ tsconfig.json  tsconfig.app.json  tsconfig.node.json
├─ .env.example    ← VITE_API_BASE=/api/v1
├─ tailwind.config.js  postcss.config.js  ← Tailwind v4 (mirror existing
pipeline)
├─ index.html
├─ src/
│   └─ main.ts                                ← app, router, pinia, axios bootstrap
├─ App.vue
├─ router/
│   └─ index.ts                                ← routes + role guards
│       └─ guards.ts                            ← auth/role middleware
├─ stores/                                    ← Pinia
│   └─ auth.ts                                ← user, isAuthenticated, login(),
logout(), changePassword()
├─ reference.ts                                ← branches, member groups, depts (cached)
├─ guest.ts                                    ← list, filter, stats, follow-ups
├─ member.ts                                  ← list, families
├─ api/
│   └─ client.ts                              ← axios instance: withCredentials,
interceptors (401→redirect)
│   └─ envelope.ts                            ← `call(area, act, content)` →
ApiResponse<T> (single chokepoint)
│   └─ auth.ts                                ← login, changePassword, logout
│   └─ data.ts                                ← getList, getListGroup
│   └─ guest.ts                               ← find, get, add, addFollowUp,
findFollowUps, complete, cancel, promote, stats, remove
│       └─ member.ts                          ← find, get, add, famAdd, famGet, famFind
├─ composables/
│   └─ useAuth.ts
│   └─ useToast.ts                            ← replaces _Alerts.cshtml
│       └─ useConfirm.ts
├─ layouts/
│   └─ DefaultLayout.vue                      ← header + sidebar + footer (replaces
_Layout.cshtml)
│       └─ AuthLayout.vue                    ← (replaces _LayoutAuth.cshtml)
├─ components/
│   └─ ui/                                    ← Button, Input, Select, Modal, Spinner,
Toast, DatePicker, DataTable
│       └─ layout/                            ← Sidebar, Topbar, BreadCrumb
│           └─ domain/
│               └─ guest/
│                   └─ GuestForm.vue            ← _AddGuest.cshtml (20+ fields,
3-col grid)
│                       └─ GuestFilters.vue
│                       └─ GuestStatCards.vue
│                       └─ FollowUpAddDialog.vue ← _AddFollowUp.cshtml
│                       └─ FollowUpCompleteDialog.vue ← _CompleteFollowUp.cshtml
│                       └─ PromoteGuestDialog.vue ← _PromoteGuest.cshtml
│                           └─ member/
│                               └─ MemberForm.vue ← _AddMember.cshtml
│                               └─ FamilyForm.vue ← _AddFam.cshtml
├─ views/                                    ← route components
├─ auth/LoginView.vue                        ← /auth/login

```

```

├── auth/ChangePassView.vue ← /auth/changepass
├── HomeView.vue             ← /
├── PrivacyView.vue         ← /privacy
├── guest/GuestListView.vue  ← /guest
├── guest/GuestRegisterView.vue ← /guest/register
├── guest/GuestDetailView.vue ← /guest/:code
├── guest/FollowUpsView.vue  ← /guest/followups
├── membership/MembersView.vue ← /membership
├── membership/MemberDetailView.vue ← /membership/member/:code
├── membership/DeptsView.vue ← /membership/depts
├── membership/FellowshipsView.vue ← /membership/flsps
├── membership/FamiliesView.vue ← /membership/fams
├── messaging/MessagingView.vue ← /messaging
├── types/                   ← re-exports from @shepherd/shared
├── assets/
│   ├── styles/main.css     ← @import "tailwindcss" + tokens lifted
│   └── from ShepherdSoft/src/css/variables.css
│       ├── images/         ← copy from
│       └── ShepherdSoft/wwwroot/{images,media}

```

Backend — Key Implementation Notes

Envelope handling (the central pattern)

Every controller in scope is a single `POST /service` that switches on `act`. Implementation:

- `ApiRequestDto` mirrors `ShepherdSoft.DBL/Models/ApiModels.cs` exactly (use `class-transformer` `@Expose({ name: 'tsp' })` etc.).
- A `CallerInterceptor` (replacing `ShepherdSoft.Api/Middlewares/WebApiDataHandler.cs`) reads the JWT cookie, parses the `userData` claim, and overwrites `body.caller` so services see a server-vouched header — never a client-supplied one.
- An `EnvelopeInterceptor` wraps thrown errors and plain return values into `ApiResponse { stat, msg, data }`.
- Services dispatch on `act`. Pattern:

```

// guest.service.ts
async handle(req: ApiRequestDto): Promise<unknown> {
  switch (req.act) {
    case HTTP_API_ACTION.GUEST_FIND:      return this.find(req.content as GuestFilter);
    case HTTP_API_ACTION.GUEST_GET:       return this.get(req.content as { code: number });
    case HTTP_API_ACTION.GUEST_ADD:       return this.add(req.content as Guest, req.caller);
    // 203..209
    default: throw new UnknownActionError(req.act);
  }
}

```

Prisma + stored procedures

1. Add a `DATABASE_URL=mysql://...` to `backend/.env`, run `pnpm prisma db pull` to generate the schema from the live DB.
2. Views (`vw_Guests`, etc.) come back as **models with no PK** — mark each with `@@ignore` and a manual `@id` on the natural key, then re-run `prisma generate`.
3. SPs are not introspected. Wrap them in typed helpers under `src/db/sp/`:

```
// guest.sp.ts
async createGuest(input: CreateGuestInput): Promise<SpResult> {
  const rows = await this.prisma.$queryRawUnsafe<SpRow[]>(
    'CALL sp_CreateGuest(?, ?, ?, /* ... 23 params */)',
    input.brCode, input.uCode, /* ... */
  );
  return rows[0]; // {resp_status, resp_message, resp_code}
}
```

The full SP list to wrap: `sp_VerifyUser`, `sp_UserLogin`, `sp_ChangeUserPwd`, `sp_CreateMember`, `sp_CreateFamily`, `sp_CreateGuest`, `sp_PromoteGuestToMember`, `sp_DeleteGuest`, `sp_AddGuestFollowUp`, `sp_CompleteGuestFollowUp`, `sp_CancelGuestFollowUp`, `bfp_GetListData`, `bfp_GetListGroup`, `fn_GenMemberNo`.

Authentication & password compat

- **Critical:** the existing password hash is produced by `SecurityManager` in `ShepherdSoft.DBL` — confirmed format is `Base64(SHA?+salt)`. Before writing `crypto.ts`, open `ShepherdSoft.DBL` and read `SecurityManager` to mirror the exact algorithm (iteration count, hash function, encoding). Seed admins are stored with this format already — getting it wrong breaks login.
- JWT issuance mirrors `TokenService.cs`: HS256, 30-min expiry, claims `sub`, `jti`, `iat`, `userData` (JSON-stringified `ApiAppModel`). Reuse the existing JWT signing key from `AppSetts.JwtToken.Key` so any unexpired tokens issued by .NET keep working during cutover.
- Login response sets `Set-Cookie: shp_jwt=<token>; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=1800` AND returns the `UserDataModel` payload (same shape as today, including `auth_token` for any code still reading the body).
- `JwtAuthGuard` reads from cookie first, falls back to `Authorization: Bearer` (helps Postman/Swagger).
- Logout = clear cookie + (optional) invalidate session row.

Caller context (replicates `WebApiDataHandler`)

For every authenticated request:

1. JWT is decoded once (Passport strategy) → attached to `req.user`.
2. `CallerInterceptor` builds a `RequestHeader` from `req.user.userData` and **overwrites** `req.body.caller`. This prevents clients from spoofing `br_code`/`ucode`.
3. `@Caller()` parameter decorator pulls the now-trusted header into handlers.

CORS, cookies, security headers

```
// main.ts
app.use(cookieParser());
app.use(helmet({ contentSecurityPolicy: false })); // CSP set explicitly below
app.enableCors({
  origin: env.ALLOWED_ORIGINS.split(','), // ['http://localhost:5173']
  credentials: true,
  methods: ['GET', 'POST'],
  allowedHeaders: ['Content-Type', 'Authorization', 'BS-XSRF-TOKEN'],
});
```

Replicates [ShepherdSoft.Api/Middlewares/SecurityHeaders.cs](#) (CSP, X-Content-Type-Options, X-Frame-Options, HSTS) via a custom middleware. Rate limit via [@nestjs/throttler](#) at 10 req / 10 s to match the existing fixed-window limiter.

Frontend — Key Implementation Notes

Single API chokepoint

[src/api/envelope.ts](#) is **the only place** that knows the envelope shape:

```
export async function call<T = unknown>(area: 'auth' | 'data' | 'guest' | 'member', act:
number, content?: unknown): Promise<T> {
  const { data } = await client.post<ApiResponse<T>>(`/${area}/service`, {
    tsp: new Date().toISOString(), ver: 1, act, content, caller: null /* server
overwrites */,
  });
  if (data.stat !== 0) throw new ApiError(data.msg, data.err_no);
  return data.data as T;
}
```

Login uses [/auth/login](#) (not [/service](#)) and is the one exception.

Axios setup

- `withCredentials: true` (cookie auth).
- Request interceptor: attach `BS-XSRF-TOKEN` from cookie (CSRF — backend issues a separate non-HttpOnly XSRF cookie that JS reads, posts back, server verifies).
- Response interceptor: on 401, clear auth store + redirect to [/auth/login](#) with `?return=<current>`.

State

- `useAuthStore` — `user`, `isAuthenticated`, `roles`, `login()`, `logout()`, `changePassword()`. Backed by a [/auth/me](#) (or first-request decoded JWT) check on app mount.
- `useReferenceStore` — caches branches, member groups, depts (loaded once after login via `DATA_GET_LIST_GROUP` for `AddGuest`, `CreateMember`).
- `useGuestStore` / `useMemberStore` — list state, filters, stats. No prefetch on visit — pages call `find()` on mount.

Forms

`vee-validate` + `zod` schemas live next to each form component (`GuestForm.vue` ships `guestSchema`). All forms map 1:1 to the existing partials (`Views/Guest/_AddGuest.cshtml`, etc.); I'll port fields verbatim — including field count, ordering, the recent 12-field expansion (feedback, born_again, sprrt_stage, grp_code), and the 3-column grid layout from commit `b12bb0e/93932dd`.

Styling

- Use **Tailwind v4** to match the source pipeline at `ShepherdSoft/postcss.config.js` and `ShepherdSoft/src/css/input.css`. Lift `variables.css` and `components.css` (design tokens + extracted KTUI classes).
- **Drop KTUI's JS layer** (`core.bundle.js`, `wwwroot/js/layouts/demo1.js`, `wwwroot/js/datatables/*`) — replace with Vue-native components. Keep KTUI's CSS classes/iconography if visually load-bearing.
- Replace DataTables.net with a lightweight Vue table (`vue3-easy-data-table` or a custom `<DataTable>` in `components/ui/`).

Database Strategy

1. **No schema changes.** The new stack connects to the same MySQL instance.
2. `pnpm prisma db pull` introspects ~14 tables + 7 views. Views need manual `@@ignore` cleanup (Prisma requires a unique key).
3. Sequences (`member_code_seq`, `guest_code_seq`, etc.) are read/incremented inside SPs — no Prisma involvement needed.
4. Add a one-off `backend/scripts/check-db.ts` that calls `bfp_GetListData(1)` to smoke-test connectivity and SP access.

Critical Files to Read Before Coding

Existing file	Why
<code>ShepherdSoft.DBL/SecurityManager.cs</code> (find via Grep)	Exact password hash + salt format. Login parity depends on this.
<code>ShepherdSoft.Api/Services/TokenService.cs</code>	JWT claim shape (especially <code>userData</code>) — frontend already consumes it.
<code>ShepherdSoft.Api/Middlewares/WebApiDataHandler.cs</code>	Caller context injection contract.
<code>ShepherdSoft.DBL/Bl.cs</code> (754 lines)	Source of truth for action-code → repo orchestration. Each <code>case</code> becomes one TS method.
<code>ShepherdSoft.DBL/Repositories/*.cs</code>	Each query maps to a Prisma view query or <code>\$queryRawUnsafe</code> SP call.

Existing file	Why
<code>ShepherdSoft.DBL/Models/ApiModels.cs + UserModels.cs</code>	Source of every DTO field — copy JSON property names exactly into <code>packages/shared</code> .
<code>ShepherdSoft.DBL/Consts/HTTP_API_ACTION.cs</code>	Action codes — must copy verbatim.
<code>ShepherdSoft/Views/Guest/_AddGuest.cshtml</code>	Guest registration UI parity (20+ fields, 3-col grid).
<code>ShepherdSoft/Views/Shared/_Layout.cshtml + _LayoutSidebar.cshtml</code>	Sidebar nav + topbar structure.
<code>Scripts/Schema Scripts.sql, Scripts/Guest Management.sql</code>	SP signatures & params — exact param order matters for <code>\$queryRawUnsafe</code> .

Phased Implementation

Phase A — Scaffolding (1 PR)

1. Create `D:\Projects\Vue\ShepherdSoft-TS\`, init pnpm workspace, root configs, `.gitignore`, `README`.
2. Create `packages/shared` with envelope types and `HTTP_API_ACTION` enum copied verbatim.
3. Create `backend` (`nest new`) and `frontend` (`pnpm create vite -- --template vue-ts`) skeletons. Verify both boot.

Phase B — Backend foundations

1. Add Prisma, `prisma db pull`, clean up view models (`@@ignore` + manual id), commit `schema.prisma`.
2. Implement `PrismaService`, SP wrapper modules under `src/db/sp/`. Smoke-test against `bfp_GetListData`.
3. Implement `crypto.ts` to match `SecurityManager` (read .NET source first; verify against a known user's stored hash).
4. Implement `auth` module: `/auth/login`, `/auth/service` (`AUTH_CHANGE_PASS`), cookie-issued JWT, `JwtAuthGuard`, `CallerInterceptor`, `EnvelopeInterceptor`, `HttpExceptionFilter`.
5. Wire CORS, helmet, `cookie-parser`, throttler, security headers middleware. Verify with curl + a real .NET-issued JWT.

Phase C — Backend modules

1. `data` module: `DATA_GET_LIST` (1), `DATA_GET_LIST_GROUP` (2). Wrap `bfp_GetListData`, `bfp_GetListGroup`.
2. `member` module: `MEMBER_FIND/GET/ADD` (100/101/102), `MEMBER_FAM_ADD/GET/FIND` (103/104/105). Calls `sp_CreateMember`, `sp_CreateFamily`, `vw_Members`, `vw_Fams`, `fn_GenMemberNo`.
3. `guest` module: `GUEST_*` (200–209). Calls `sp_CreateGuest`, `sp_AddGuestFollowUp`, `sp_CompleteGuestFollowUp`, `sp_CancelGuestFollowUp`, `sp_PromoteGuestToMember`, `sp_DeleteGuest`, `vw_Guests`, `vw_GuestFollowUps`. **Stats endpoint** (`GUEST_GET_STATS`) is the only one with a custom aggregate query — port the SQL from `GuestRepository.cs` verbatim.
4. Per-module e2e tests against a test DB (seeded with `Data Scripts.sql`).

Phase D — Frontend foundations

1. Router, Pinia, axios client, envelope wrapper, auth store, CSRF cookie handshake.
2. `AuthLayout` + `LoginView` + `ChangePassView`. Verify cookie-based login round-trip end-to-end.
3. `DefaultLayout` (sidebar + topbar). Port nav structure from `_LayoutSidebar.cshtml`.

Phase E — Frontend features

1. `HomeView` (dashboard with `GuestStatCards` — calls `GUEST_GET_STATS`).
2. Guest flows: `GuestListView` (filters + table), `GuestRegisterView` (modal/page reuse `GuestForm.vue`), `GuestDetailView`, `FollowUpsView` with add/complete/cancel dialogs and `PromoteGuestDialog`.
3. Membership flows: `MembersView`, `MemberDetailView`, `FamiliesView`, `DeptsView`, `FellowshipsView`.
4. `MessagingView` (stub if backend module is empty in .NET — confirm during Phase A).

Phase F — Parity verification + cutover

1. Side-by-side smoke test: log in as the same user against both stacks, run every action, diff responses.
2. Update `ShepherdSoft.Api/appsettings.json` `AllowedOrigins` if the SPA deploys to a different host.
3. When parity is signed off: stop deploying `ShepherdSoft.Api` for branch-user traffic; redirect/proxy `/api/v1/auth|data|guest|member` to NestJS, keep `/api/v1/admin/*` on .NET.

Assumptions & Flags for Manual Review

1. **SecurityManager algorithm** — assumed PBKDF2-HMAC-SHA256 with the stored salt; **verify by reading the file before writing `crypto.ts`**. Existing seed admins use hash `B1FMnzth5r/N/UEiAf0GMoM/VZ4zgi0Yu0Twjip9EN0=` + salt `wYxa3jlohehTIIu8` — test against these.
2. **Sessions table** — `sp_UserLogin` writes a row to `user_sessions`. Need to confirm whether session validity is checked on every request or only at login. If yes, NestJS must call the SP on each request (expensive) or shift to pure-JWT and accept that the .NET admin will keep its own session tracking.
3. **MessagingView** — controller exists in MVC but no corresponding `.Api` controller in scope. Confirm whether messaging is a planned feature with no backend yet, or whether it routes elsewhere. Stub the view.
4. **KTUI license** — `@keenthemes/ktui` is a paid theme. Confirm the license permits use in the new Vue project before reusing classes/iconography wholesale.
5. **CSRF** — current MVC uses `BS-XSRF-TOKEN` antiforgery; current `.Api` does not (it's pure JWT). With cookie auth on the SPA, CSRF protection becomes necessary — adding a double-submit cookie pattern. Confirm acceptable.
6. **Admin/branch-user data overlap** — admin .NET API still reads/writes the same tables. Concurrent writes are safe (DB-level constraints), but cache invalidation in the new frontend reference store is best-effort — confirm acceptable.
7. **AllowedOrigins** — in production the SPA may be served by NestJS itself (`serve-static`) on the same origin, eliminating CORS. Decide before deploy.
8. **TypeScript strict mode** — `strict: true`, `noUncheckedIndexedAccess: true`, ESLint `@typescript-eslint/no-explicit-any: error`. No *any*, ever, per the prompt.

Verification

Local smoke (after Phase B):

```
# In backend/
pnpm prisma generate && pnpm start:dev
# In another shell — should return {stat:0, data:{...}}
curl -X POST http://localhost:3000/api/v1/data/service `
  -H "Content-Type: application/json" `
  -d '{"tsp":"now","ver":1,"act":1,"content":{"typ":1,"code":0}}'
```

Auth smoke (after Phase B):

```
curl -i -c cookies.txt -X POST http://localhost:3000/api/v1/auth/login `
  -H "Content-Type: application/json" `
  -d '{"tsp":"now","ver":1,"act":0,"content":{"uname":"<seeded-user>","pwd":"<known>","br_code":1}}'
# expect Set-Cookie: shp_jwt=...; HttpOnly
curl -b cookies.txt -X POST http://localhost:3000/api/v1/guest/service `
  -H "Content-Type: application/json" `
  -d '{"tsp":"now","ver":1,"act":208,"content":{}}'
# expect guest stats
```

Parity test (after Phase E): Run both stacks. Same login. For each action code:

1. Hit the .NET endpoint with the .NET-issued JWT.
2. Hit the NestJS endpoint with the NestJS-issued JWT (same user).
3. Diff JSON responses — only `tsp/jti`/login timestamps may differ.

End-to-end (after Phase F):

1. `pnpm dev` in `frontend/` + `pnpm start:dev` in `backend/`.
2. Login, navigate every route (`/`, `/guest`, `/guest/register`, `/guest/:code`, `/guest/followups`, `/membership`, `/membership/depts`, `/membership/flsps`, `/membership/fams`, `/messaging`, `/auth/changepass`).
3. Register a guest, schedule + complete a follow-up, promote to member. Verify rows in MySQL match .NET flow exactly.
4. Run e2e specs (`pnpm test:e2e`) — 100% pass.

Definition of Done

- All in-scope action codes pass byte-for-byte response parity vs. .NET (excluding timestamps).
- Login round-trips through `HttpOnly` cookie; logout clears it; 401s redirect to `/auth/login`.
- `tsc --noEmit` clean on both projects; `eslint` . clean; `pnpm test` green.
- README documents: prerequisites (Node 20+, pnpm, MySQL access), env setup, `pnpm dev` per side, deployment notes.
- `.env.example` files committed for both `backend` and `frontend`; real `.env` files git-ignored.
- The .NET projects remain buildable and deployable for the Admin Area.